PowerMe
Hiểu dự án từ số 0

Tổng quan dự án, giải thích OCPP cho người chưa từng đọc spec, và lộ trình những gì bạn còn phải học — xếp theo thứ tự cần dùng, không theo thứ tự sách giáo khoa.

NGƯỜI ĐỌC
Ngô Văn Minh Trí · SE191045

VAI TRÒ
WP2 owner · Team Leader

PHẦN PHỤ TRÁCH
CSMS OCPP 1.6J · Đối soát

NGÀY SOẠN
12/09/2026

> **Cách dùng tài liệu này — đọc trước khi đọc tiếp**
>
> Đây là tài liệu **để hiểu**, không phải tài liệu **để trích dẫn**. Mọi số hiệu mục trong spec, mọi giá trị enum, mọi hành vi cụ thể của firmware — phải đối chiếu lại với bản OCPP 1.6 gốc và JSON schema trước khi đưa vào Report hay khẳng định trước hội đồng.
>
> Lý do không phải hình thức: dự án này đã nhiều lần phải sửa kết luận vì tin nguồn thứ cấp. Chính tài liệu này cũng là nguồn thứ cấp. Xem mục 07.

01

Dự án này thực ra là gì

Bài toán có thật

Một doanh nghiệp mua trụ sạc xe điện DC nhập khẩu — dòng **D1 Series** và **YJ-EU Series**, 60–160 kW, súng đôi, của hãng Jiangxi Ruihua (Trung Quốc). Mỗi trụ đi kèm phần mềm quản lý mặc định của hãng. Phần mềm đó là hộp đen: không sửa được, không tích hợp được với hệ thống thanh toán Việt Nam, không xuất được dữ liệu theo cách doanh nghiệp cần.

PowerMe thay thế lớp phần mềm đó. Trụ vẫn là trụ của hãng; cái nhóm xây là phần "bộ não" ở phía server nói chuyện với trụ, cộng app cho tài xế và dashboard cho người vận hành.

> **Vì sao thay được mà không cần đụng vào phần cứng**
>
> Vì trụ nói một **giao thức mở** tên là OCPP. Bất kỳ server nào nói đúng giao thức đó đều điều khiển được trụ. Kỹ thuật viên chỉ cần vào màn hình HMI của trụ, gõ lại địa chỉ server trỏ về PowerMe thay vì về server của hãng, rồi khởi động lại. Toàn bộ dự án đứng được là nhờ tính chất này của OCPP.

Bốn thành phần

THÀNH PHẦN	WP	LÀM GÌ
CSMS	**WP2 — bạn**	Charging Station Management System. Server nói OCPP với trụ, giữ trạng thái từng súng sạc, ghi nhận giao dịch, đối soát số liệu. **Đây là lõi kỹ thuật của đồ án.**
App tài xế	WP3	Tìm trạm, quét QR bắt đầu sạc, xem tiến trình, thanh toán VNPay
Dashboard	WP4	Người vận hành theo dõi trụ, xử lý sự cố, quản trị
Billing + Simulator	WP5	Tính tiền, loyalty, và trụ ảo để kiểm thử

> ### Ràng buộc lớn nhất: không có trụ vật lý
>
> Nhóm không có trụ thật để cắm vào. Toàn bộ kiểm thử chạy trên **simulator** — một chương trình giả làm trụ sạc, nói đúng OCPP nhưng không có điện. Đây là rủi ro đã chấp nhận (mã `R-H01`), không phải thiếu sót.
>
> Hệ quả phải nhớ suốt dự án: **simulator trả lời theo cách người viết simulator nghĩ trụ sẽ trả lời.** Mọi câu hỏi dạng "firmware thật gửi gì" — simulator không trả lời được. Phải hỏi hãng.

Câu hỏi nghiên cứu — thứ phân biệt đồ án với một sản phẩm

Một đồ án Capstone không chỉ cần chạy được, mà cần trả lời một câu hỏi chưa có sẵn đáp án. Câu hỏi của bạn:

> **Một CSMS tự xây đối soát dữ liệu giao dịch và đo đếm *chính xác tới đâu* khi đường truyền tới trụ chập chờn?**

Mục 04 giải thích vì sao câu này khó và vì sao nó đáng hỏi. Cần hiểu mục 02 trước.

02

OCPP, giải thích từ số 0

2.1 · Vấn đề OCPP sinh ra để giải

Trước khi có chuẩn chung, mỗi hãng trụ sạc dùng giao thức riêng. Doanh nghiệp mua trụ của ba hãng thì phải chạy ba phần mềm quản lý, không gộp được. Đổi hãng nghĩa là vứt bỏ toàn bộ phần mềm.

OCPP — Open Charge Point Protocol, do tổ chức **Open Charge Alliance (OCA)** ban hành — là bộ quy ước chung: trụ và server nói chuyện bằng những bản tin có tên và cấu trúc cố định. Hãng nào cũng hiểu.

Phiên bản dự án dùng là **OCPP 1.6**, bản phổ biến nhất trong thực tế hiện nay. **2.0.1** mới hơn, vá được vài điểm yếu của 1.6 (mục 4.2) nhưng trụ Ruihua chỉ hỗ trợ 1.6 — nên 1.6 không phải lựa chọn, mà là bắt buộc.

2.2 · Hai nhân vật, và ai gọi ai

TÊN TRONG SPEC	TRONG DỰ ÁN	VAI
Charge Point (CP)	Trụ sạc D1 / YJ-EU (hoặc simulator)	Thiết bị ngoài hiện trường. Cắm điện vào xe, đo điện năng, có màn hình HMI
Central System (CS)	**CSMS của bạn**	Server. Biết mọi trụ, lưu mọi giao dịch, ra lệnh cho trụ

Điểm dễ nhầm nhất với người mới

Trụ là bên chủ động kết nối, không phải server. Trụ nằm sau router 4G hoặc mạng nội bộ, thường không có IP công khai — server không thể "gọi" trụ trước được. Nên trụ tự mở kết nối tới server và **giữ kết nối đó mở liên tục**. Mọi lệnh server gửi xuống đều đi qua ống đã mở sẵn đó.

Vì vậy: **mất kết nối là mất cả hai chiều.** Không chỉ server không biết trụ đang làm gì, mà server cũng không ra lệnh được. Đây là gốc rễ của câu hỏi nghiên cứu.

2.3 · Chữ "J" trong 1.6J

OCPP 1.6 có hai cách truyền: **SOAP** (cũ, nặng, XML) và **JSON over WebSocket** — viết tắt là **OCPP-J**, chữ J đó. Dự án dùng bản J.

- **WebSocket** là kiểu kết nối giữ mở hai chiều, khác HTTP thông thường (hỏi – đáp rồi đóng). Khi đã mở, cả hai bên đều gửi được bất cứ lúc nào.
- Trụ kết nối tới URL dạng `wss://server/ocpp/<định-danh-trụ>`. **Đoạn cuối URL chính là danh tính của trụ** — server đọc đoạn đó để biết trụ nào vừa vào. Trong dự án này, định danh là **số serial** của trụ (ADR-0009).
- Có trường `Sec-WebSocket-Protocol` báo phiên bản, giá trị `ocpp1.6`.
- `wss://` là bản mã hoá TLS. ADR-0011 chọn Security Profile 2 = TLS + Basic Auth riêng từng trụ.

2.4 · Hình dạng một bản tin

Mọi bản tin OCPP-J là **một mảng JSON**. Phần tử đầu tiên là con số cho biết đây là loại gì.

SỐ	TÊN	CẤU TRÚC
`2`	**CALL**	`[2, "id", "TênHànhĐộng", { tham số }]` — một yêu cầu
`3`	**CALLRESULT**	`[3, "id", { kết quả }]` — trả lời thành công
`4`	**CALLERROR**	`[4, "id", "MãLỗi", "Mô tả", { chi tiết }]` — trả lời lỗi

`"id"` là chuỗi do bên gửi tự đặt, duy nhất. Bên trả lời **phải chép lại đúng id đó** — đó là cách ghép câu trả lời với câu hỏi khi nhiều bản tin đan xen nhau.

```
→ [2,"19223201","BootNotification",{"chargePointVendor":"Ruihua",
     "chargePointModel":"D1-120","chargePointSerialNumber":"RH24D1120001"}]

← [3,"19223201",{"status":"Accepted","currentTime":"2026-09-12T03:14:07Z",
     "interval":300}]
```

Đọc được đoạn trên là đọc được OCPP. Phần còn lại chỉ là học tên các hành động và ý nghĩa tham số.

> **Luật một-CALL-một-lúc**
>
> Mỗi chiều chỉ được có **một CALL đang chờ trả lời**. Gửi CALL thứ hai khi cái thứ nhất chưa có CALLRESULT là sai spec. Đây là lý do thư viện `ocpp-rpc` đặt `callConcurrency = 1` mặc định, với chú thích ngay trong mã nguồn: *"OCPP requires this to be set to 1"*.
>
> Nhưng luật này chỉ ràng buộc **bên gửi**. Nó không ngăn trụ bắn liên tiếp nhiều CALL vào server. Xử lý phía nhận vẫn phải tự tuần tự hoá — xem mục 05, tầng 2.

2.5 · Các bản tin cần biết

OCPP 1.6 chia bản tin thành **profile** (nhóm chức năng): Core, Firmware Management, Local Auth List, Reservation, Smart Charging, Remote Trigger. Dự án tập trung **Core** — nhóm bắt buộc mọi trụ phải hỗ trợ.

TRỤ → SERVER	NGHĨA
`BootNotification`	"Tôi vừa khởi động, đây là model / serial của tôi". Server trả về nhịp heartbeat
`Heartbeat`	"Tôi còn sống". Gửi định kỳ khi không có gì khác để nói
`StatusNotification`	"Súng số N vừa đổi trạng thái". Bản tin quan trọng nhất để theo dõi trụ
`Authorize`	"Thẻ này có được sạc không?"
`StartTransaction`	"Bắt đầu sạc rồi, chỉ số đồng hồ lúc này là X"
`MeterValues`	"Số đo hiện tại: điện năng, dòng, áp, phần trăm pin xe…"
`StopTransaction`	"Dừng rồi, chỉ số cuối là Y, lý do dừng là Z"
`DataTransfer`	Ống thoát cho dữ liệu riêng của hãng, ngoài chuẩn

SERVER → TRỤ	NGHĨA
`RemoteStartTransaction`	"Bắt đầu sạc súng N cho thẻ này" — đường của app QR
`RemoteStopTransaction`	"Dừng giao dịch số N" — cách CSMS thực thi hạn mức
`ChangeConfiguration` / `GetConfiguration`	Đọc và ghi cấu hình của trụ
`Reset`	Khởi động lại trụ (mềm hoặc cứng)
`UnlockConnector`	Nhả khoá súng khi bị kẹt
`ChangeAvailability`	Đưa trụ / súng vào trạng thái ngừng phục vụ
`ClearCache`	Xoá bộ nhớ đệm thẻ đã xác thực trong trụ

2.6 · Sáu từ vựng phải thuộc

chargePointId	Định danh một trụ. Lấy từ đoạn cuối URL WebSocket. Dự án dùng số serial.
connectorId	Số hiệu súng sạc *trong* một trụ. `0` = **cả trụ** · `1` = **súng A** · `2` = **súng B.** Khoá nghiệp vụ của hệ thống là cặp `(chargePointId, connectorId)`, không bao giờ chỉ riêng trụ.
idTag	Chuỗi định danh người sạc. Có thể là số thẻ RFID, hoặc mã do app sinh ra khi quét QR. Đây là thứ OCPP dùng để hỏi "ai đang sạc".
transactionId	Số nguyên **do server cấp** khi trả lời `StartTransaction`. Mọi bản tin sau đó của phiên sạc đều mang số này. Trụ không tự đặt.
meterStart / meterStop	Chỉ số công tơ tích luỹ của súng, lúc bắt đầu và lúc kết thúc. **Đơn vị là Wh, không phải kWh.** Điện năng bán ra = hiệu hai số.
measurand	Loại đại lượng trong `MeterValues`. Mặc định khi không ghi rõ là `Energy.Active.Import.Register` (Wh). Ngoài ra có `SoC` (% pin xe), `Voltage`, `Current.Import`, `Power.Active.Import`, `Temperature`.

2.7 · Một phiên sạc trọn vẹn

Đây là phần nên đọc kỹ nhất trong tài liệu này. Kịch bản: tài xế quét QR trên app.

TRỤ → SERVER · LÚC TRỤ MỚI BẬT

BootNotification

Trụ báo danh. Server trả `status: "Accepted"`, kèm `currentTime` (để trụ chỉnh đồng hồ) và `interval: 300` (gửi heartbeat mỗi 300 giây).

TRỤ → SERVER

StatusNotification · Available

Mỗi súng báo trạng thái rồi. Từ lúc này server biết trụ sẵn sàng.

SERVER → TRỤ · SAU KHI TÀI XẾ BẤM NÚT TRÊN APP

RemoteStartTransaction

```
→ [2,"cs-8891","RemoteStartTransaction",
     {"connectorId":1,"idTag":"APP-USER-10237"}]
← [3,"cs-8891",{"status":"Accepted"}]
```

Bẫy cho người mới: `"Accepted"` ở đây chỉ nghĩa là *trụ đã nhận lệnh*. Nó **không** nghĩa là phiên sạc đã bắt đầu. Phiên chỉ thực sự bắt đầu khi trụ gửi `StartTransaction` ở bước 6. Nhầm hai cái này là app báo "đang sạc" trong khi xe chưa cắm.

TRỤ → SERVER

StatusNotification · Preparing

Trụ chuyển sang chờ cắm súng.

TRỤ → SERVER

Authorize

```
→ [2,"19223203","Authorize",{"idTag":"APP-USER-10237"}]
← [3,"19223203",{"idTagInfo":{"status":"Accepted"}}]
```

`idTagInfo.status` có thể là `Accepted`, `Blocked`, `Expired`, `Invalid`, `ConcurrentTx`.

TRỤ → SERVER · ĐÂY MỚI LÀ ĐIỂM BẮT ĐẦU THẬT

StartTransaction

```
→ [2,"19223204","StartTransaction",{"connectorId":1,
     "idTag":"APP-USER-10237","meterStart":1842300,
     "timestamp":"2026-09-12T03:14:31Z"}]
← [3,"19223204",{"transactionId":80412,
     "idTagInfo":{"status":"Accepted"}}]
```

`meterStart: 1842300` là **1.842,3 kWh** — công tơ tích luỹ cả đời của súng, tính bằng Wh. Server cấp `transactionId: 80412`.

TRỤ → SERVER · LẶP LẠI SUỐT PHIÊN

MeterValues

```
→ [2,"19223205","MeterValues",{"connectorId":1,"transactionId":80412,
    "meterValue":[{"timestamp":"2026-09-12T03:15:31Z","sampledValue":[
      {"value":"1843250","measurand":"Energy.Active.Import.Register","unit":"Wh"},
      {"value":"62","measurand":"SoC","unit":"Percent"},
      {"value":"398.4","measurand":"Voltage","unit":"V"},
      {"value":"118.6","measurand":"Current.Import","unit":"A"}]}]}]
```

Đây là nguồn dữ liệu cho thanh tiến trình trong app, và là **dữ liệu gốc cho engine đối soát**.

TRỤ → SERVER

StopTransaction

```
→ [2,"19223260","StopTransaction",{"transactionId":80412,
    "meterStop":1871400,"timestamp":"2026-09-12T03:47:02Z",
    "reason":"Remote"}]
```

Điện năng bán = 1.871.400 − 1.842.300 = **29.100 Wh = 29,1 kWh**. Đây là con số ra hoá đơn.

TRỤ → SERVER

StatusNotification · Finishing → Available

Súng nhả, trụ về rỗi. Vòng đời khép lại.

2.8 · Chín trạng thái của một súng sạc

`StatusNotification` báo súng đang ở trạng thái nào. Toàn bộ state machine phải xây trong CSMS xoay quanh chín giá trị này:

TRẠNG THÁI	NGHĨA
`Available`	Rỗi, sẵn sàng
`Preparing`	Đã nhận lệnh / đã quẹt thẻ, đang chờ cắm súng
`Charging`	Đang truyền điện
`SuspendedEV`	Tạm dừng vì **xe** không nhận nữa (pin đầy, BMS ngắt)
`SuspendedEVSE`	Tạm dừng vì **trụ** không cấp (quá tải, quá nhiệt)
`Finishing`	Đã dừng, chờ rút súng
`Reserved`	Đã đặt trước ~~`NGOÀI PHẠM VI`~~
`Unavailable`	Chủ động ngừng phục vụ
`Faulted`	Lỗi

Phân biệt `SuspendedEV` và `SuspendedEVSE` là một trong những chỗ hay bị hỏi khi bảo vệ, vì nó cho thấy bạn hiểu trụ hay chỉ chép spec. **EV = xe từ chối. EVSE = trụ từ chối.**

03

OCPP của dự án này khác sách giáo khoa ở đâu

Sáu đặc điểm của phần cứng Ruihua, đọc từ manual gốc. Mỗi cái đẻ ra một yêu cầu kỹ thuật mà tài liệu OCPP chung không nói.

ĐẶC ĐIỂM PHẦN CỨNG	HỆ QUẢ PHẢI XỬ LÝ
Một trụ = hai súng	Hai giao dịch hoàn toàn độc lập chạy song song *trên cùng một kết nối WebSocket*. Mọi bảng dữ liệu, mọi state machine phải khoá theo `(chargePointId, connectorId)`
Trụ có bảng giá riêng (Sharp / Peak / Shoulder / Off-peak, đồng bộ NTP) và tự hiện màn Settlement	OCPP 1.6J **không có lệnh chuẩn để đẩy giá xuống trụ**. Nghĩa là trụ và CSMS có thể tính ra hai số tiền khác nhau. Quyết định: **CSMS là nguồn sự thật** cho bản ghi phiên và bản tính giá
Ba đường xác thực: QR qua app · RFID · mật khẩu trên HMI	Chỉ đường QR đi qua `RemoteStartTransaction`. Hai đường kia trụ tự gửi `Authorize` — **phiên mở ra mà app hoàn toàn không biết**. CSMS phải xử lý được phiên "từ trên trời rơi xuống"
Bốn chế độ sạc	Chỉ chế độ "sạc đầy tự động" là trụ tự dừng. Các chế độ theo thời gian / theo kWh / theo số tiền đều buộc **CSMS phải theo dõi `MeterValues` rồi chủ động bắn `RemoteStopTransaction`**. Nếu CSMS mất kết nối đúng lúc đó, hạn mức không được thực thi
"Wallet ID card" — thẻ trả trước, **trụ tự trừ tiền trên thẻ kể cả khi offline**	Nếu billing thu tiếp qua VNPay thì **khách bị trừ hai lần**. Giải pháp (ADR-0012): tách `origin` (phiên bắt đầu thế nào) khỏi `payment_mode` (ai thu tiền). Phiên `station_wallet` vẫn tính giá để đối soát nhưng không bao giờ thu qua VNPay
Kết nối Ethernet hoặc router 4G	Mất kết nối là chuyện thường ngày, không phải trường hợp hiếm. Đây là **động cơ có thật** cho câu hỏi nghiên cứu, không phải tình huống bịa ra

Hai chỗ dễ hiểu sai trong manual Ruihua

"plug-and-charge" trong manual nghĩa là *cắm vào là sạc, không cần xác thực*. Đây **không phải** Plug & Charge theo chuẩn ISO 15118 (chứng thư số trong xe). Trong mọi tài liệu của nhóm, gọi tính năng này là **free vend**.

Mật khẩu admin mặc định của HMI là `115200` — đây là một phát hiện bảo mật, nên đưa vào báo cáo như một finding, không bỏ qua.

04

Câu hỏi nghiên cứu, giải thích cho người mới

4.1 · Chuyện gì xảy ra khi trụ mất mạng giữa phiên sạc

Xe vẫn đang sạc — điện vẫn chảy, công tơ vẫn quay. Chỉ có đường truyền tới server là đứt. Trụ không vứt bản tin đi; nó **xếp hàng trong bộ nhớ** và gửi dồn khi nối lại được.

Nghe thì ổn. Vấn đề nằm ở phía server.

4.2 · Server không phân biệt được "vừa xảy ra" với "xảy ra 20 phút trước"

Khi trụ nối lại, server nhận một loạt bản tin dồn về. Trong đó có thể lẫn cả bản tin thời gian thực. Server cần biết cái nào là cái nào — để tính tiền đúng, để hiển thị đúng, để không tạo giao dịch trùng.

> **Khoảng trống đã xác minh trong spec**
>
> OCPP 1.6 **thừa nhận** rằng Central System **không có dấu hiệu nào ở tầng giao thức** để phân biệt bản tin phát lại từ hàng đợi offline với bản tin thời gian thực. Server chỉ có thể **suy đoán qua timestamp** — mà timestamp lại do đồng hồ của trụ sinh ra, và đồng hồ trụ có thể lệch.
>
> OCA đã vá lỗ hổng này ở **2.0.1** bằng cách thêm trường `offline` và `seqNo`. Nhưng trụ của dự án chỉ chạy 1.6.

Đây chính là chỗ đóng góp của bạn nằm: **xây một engine đối soát bù đắp cho thứ giao thức không cung cấp**, rồi **đo xem nó chính xác tới đâu**.

4.3 · Vì sao thiết kế hệ thống phải theo câu hỏi này

Ba quyết định kiến trúc sinh trực tiếp từ đây, và phải giải thích được khi bị hỏi:

ADR	QUYẾT ĐỊNH VÀ LÝ DO
0005	**Ghi nhật ký bản tin OCPP *trước khi* xử lý**, vào bảng chỉ-thêm-không-sửa. Vừa chống mất dữ liệu khi xử lý lỗi, vừa tạo ra **ground truth** — bộ dữ liệu gốc để so sánh trong thí nghiệm. Không có nó thì không đo được gì
0009	Lưu **cả hai** mốc thời gian: `cp_timestamp` (đồng hồ trụ — không tin được) và `received_at` (đồng hồ server — tin được). Chênh lệch giữa hai cái chính là **tín hiệu để phát hiện bản tin phát lại**
0006	**Không dùng Redis pub/sub trên đường đi của giao dịch.** Pub/sub mất bản tin khi không có subscriber — tức là nó sẽ *tự gây ra* đúng loại mất dữ liệu mà đề tài đang đo. Pub/sub chỉ dùng cho cập nhật giao diện

> **Điều phải trung thực khi viết Report**
>
> Khoảng trống trong spec **không tự động là tính mới học thuật**. Có thể đã có người công bố về vấn đề này. Trước khi tuyên bố đóng góp, phải khảo sát IEEE / ScienceDirect / Google Scholar. *"Không tìm được bằng chứng"* khác *"đã chứng minh là chưa ai làm"* — viết đúng cái mình có.

05

Còn phải học gì — xếp theo thứ tự cần dùng

Xếp theo thứ tự **sẽ dùng tới**, không theo thứ tự sách giáo khoa. Nguyên tắc xuyên suốt: **chạy trước, đọc sau.** Đọc spec khi chưa thấy bản tin thật chạy qua màn hình thì chữ không dính.

TẦNG 0 Nền tảng — thiếu cái này thì mọi thứ sau đều mơ hồ

~4–5 ngày

BẮT BUỘC WebSocket, đủ sâu để gỡ lỗi

- Khác gì HTTP: bắt tay một lần rồi giữ mở, hai chiều, không có khái niệm "request thất bại" như HTTP
- **Ping / pong** ở tầng giao thức — cách phát hiện kết nối chết mà chưa đóng. `ocpp-rpc` đặt `pingIntervalMs = 30000`
- **Half-open connection:** một bên tưởng còn kết nối, bên kia đã mất. Đây là tình huống thật của trụ 4G và là nguồn của nhiều lỗi khó tái hiện
- Reconnect và backoff. *Lưu ý: spec OCPP quy định backoff **tuyến tính**, nhiều nguồn thứ cấp viết nhầm thành exponential — kiểm lại spec gốc trước khi trích*

BẮT BUỘC State machine

- Trạng thái, chuyển trạng thái, chuyển trạng thái hợp lệ và không hợp lệ. Bạn sẽ viết state machine cho 9 trạng thái ở mục 2.8, nhân hai súng. Vẽ ra giấy trước khi viết code

BẮT BUỘC Thời gian trong hệ phân tán

- UTC, múi giờ, kiểu `timestamptz` của PostgreSQL
- **Lệch đồng hồ (clock skew):** vì sao không bao giờ được tin đồng hồ của thiết bị ngoài hiện trường
- Vì sao ADR-0009 lưu cả hai mốc thời gian — hiểu lý do, không chỉ nhớ quy tắc

BẮT BUỘC SQL và PostgreSQL ở mức vận hành

- Transaction, `COMMIT` / `ROLLBACK`, ý nghĩa của isolation level
- **Ràng buộc `UNIQUE` nhiều cột** — cơ chế thực thi idempotency của ADR-0010
- Chỉ mục, và vì sao `MeterValues` cần partition theo tháng (ADR-0007)

TẦNG 1 OCPP — chỉ đọc phần sẽ dùng

~5–7 ngày

Spec OCPP 1.6 rất dài. Đọc tuần tự từ đầu là cách chắc chắn nhất để bỏ cuộc. Chia ba mức:

PHẢI THUỘC

- Cấu trúc CALL / CALLRESULT / CALLERROR và quy tắc `uniqueId`
- Toàn bộ **Core Profile** — bảng ở mục 2.5, cả hai chiều
- Chín trạng thái connector và các chuyển đổi hợp lệ
- Cấu trúc `MeterValues` : `sampledValue` , `measurand` , `unit` , `context`
- **Hành vi offline:** hàng đợi bản tin, và bốn khoá cấu hình `LocalAuthorizeOffline` , `LocalPreAuthorize` , `TransactionMessageAttempts` , `TransactionMessageRetryInterval`

CHỈ CẦN TRA ĐƯỢC KHI DÙNG

- Danh sách mã lỗi CALLERROR · Firmware Management · Local Auth List · Remote Trigger · `DataTransfer`

~~BỎ HẲN~~

- SOAP binding · Smart Charging profile · Reservation · toàn bộ OCPP 2.0.1 (chỉ đọc đúng phần `offline` / `seqNo` để trích trong chương nghiên cứu)

> **Mẹo đọc nhanh gấp nhiều lần:** đọc **JSON schema** của từng bản tin thay vì đọc văn xuôi. Schema cho biết chính xác trường nào bắt buộc, kiểu gì, giá trị enum nào hợp lệ — trong 20 dòng thay vì 3 trang. Thư mục `ocppdl/` trong repo có sẵn schema để tra *(chỉ để tra, không phải dependency)*.

TẦNG 2 Ngăn xếp kỹ thuật của CSMS

~7–10 ngày

BẮT BUỘC TypeScript strict + NestJS

- TypeScript ở chế độ strict (ADR-0003): kiểu rời rạc, narrowing, vì sao `any` làm hỏng giá trị của `packages/shared`
- NestJS: module, provider, dependency injection, vòng đời. Đủ để hiểu vì sao ADR-0002 chọn **modular monolith** — một tiến trình, ranh giới module rõ ràng, không microservices

BẮT BUỘC `ocpp-rpc` và ba việc nó không làm

Thư viện lo tầng vận chuyển: đóng gói bản tin, khớp `uniqueId`, parse Basic Auth, validate schema, timeout 60s, ping 30s. **Ba việc nó không làm và bạn phải tự viết:**

1. **Tuần tự hoá xử lý theo từng trụ.** Thư viện không giới hạn đồng thời cho CALL *chiều vào*. Nếu xử lý song song, bản ghi vào DB sai thứ tự và dữ liệu đối soát hỏng — đúng thứ đồ án đang đo
2. **Trụ nối lại khi phiên cũ còn treo.** Cùng một định danh mở kết nối mới trong khi kết nối cũ chưa đóng hẳn. Phải tự phát hiện và đóng phiên cũ
3. **Toàn bộ logic nghiệp vụ OCPP.** Thư viện không biết `StartTransaction` nghĩa là gì

NÊN CÓ Drizzle ORM · Docker Compose · kiểm thử

- Drizzle: schema bằng TypeScript, migration (ADR-0008 — *đang chờ chốt cùng WP5*)
- Docker Compose: đã có sẵn trong repo. Cần hiểu đủ để sửa, không cần thành thạo
- Kiểm thử code giao thức: dựng bản tin giả, kiểm tra state machine, kiểm tra idempotency. Đây là phần dễ bị bỏ và bị hỏi khi bảo vệ

TẦNG 3 Phương pháp nghiên cứu — phần nâng điểm

~3–4 ngày, song song

- **Thiết kế thí nghiệm:** biến độc lập (thời điểm và độ dài lần ngắt kết nối), biến phụ thuộc (sai lệch điện năng, giao dịch trùng, giao dịch mất), nhóm đối chứng (phiên không bị ngắt)
- **Ground truth:** lấy đâu ra "đáp án đúng" để so? Chính là log bản tin thô của ADR-0005 cộng với log phía simulator
- **Chỉ số đo:** định nghĩa trước khi chạy, không định nghĩa sau khi nhìn kết quả
- **Tính tái lập:** script chạy lại được, dữ liệu thô giữ nguyên, mô tả đủ để người khác làm lại
- **Khảo sát tài liệu:** IEEE Xplore, ScienceDirect, Google Scholar — trước khi tuyên bố đóng góp (mục 4.3)

> Phần này **không phụ thuộc ngôn ngữ backend**. Script phân tích dữ liệu viết bằng Python (pandas / matplotlib) hoàn toàn được, dù CSMS chạy Node.

Việc đầu tiên nên làm, trước cả khi học xong tầng 0

Dựng bàn thí nghiệm OCPP. SteVe (một CSMS mã nguồn mở, dùng làm mốc đối chiếu) + SAP simulator + một WebSocket proxy khoảng 50 dòng đứng giữa, ghi lại mọi frame đi qua.

Rồi chạy một phiên sạc, và **tự tay vẽ lại sequence diagram từ log của chính mình** — không copy từ tài liệu nào. Đây là bài tập đổi nhiều giờ đọc spec lấy một buổi chiều, và là cách duy nhất để những thứ ở mục 2.7 chuyển từ "đã đọc" sang "đã hiểu".

Sau đó ngắt kết nối theo ba kịch bản: **giữa chuỗi MeterValues · ngay trước StopTransaction · mở phiên mới khi đang offline**. Ghi lại log thô. Đó là dữ liệu nền cho chương nghiên cứu.

06

Tự kiểm — trả lời được 14 câu này là đã hiểu

Không tra tài liệu. Câu nào ngắc ngứ thì đó là chỗ cần quay lại.

01 Vì sao trụ là bên mở kết nối, không phải server? Hệ quả gì khi mất mạng?

02 Trong `[2,"abc","Authorize",{…}]`, mỗi phần tử nghĩa là gì? Nếu số đầu là `3` thì cấu trúc khác thế nào?

03 `connectorId = 0` nghĩa là gì? Vì sao khoá nghiệp vụ không thể chỉ là `chargePointId`?

04 Ai cấp `transactionId` — trụ hay server? Vào thời điểm nào?

05 `meterStart` đơn vị gì? Nhầm đơn vị thì hoá đơn sai bao nhiêu lần?

06 Trụ trả `RemoteStartTransaction` = `Accepted`. Xe đã bắt đầu sạc chưa? Làm sao biết chắc?

07 Phân biệt `SuspendedEV` và `SuspendedEVSE`. Cho một tình huống thật của mỗi loại.

08 Tài xế quẹt RFID thay vì dùng app. App có biết phiên này không? Vì sao?

09 Trụ mất mạng giữa phiên rồi nối lại. Server nhận được gì, và vì sao không phân biệt được bản tin nào là phát lại?

10 Vì sao ADR-0010 khoá `UNIQUE` trên `(charge_point_id, connector_id, id_tag, meter_start_wh, cp_timestamp)`? Không có nó thì hỏng thế nào?

11 Vì sao lưu cả `cp_timestamp` lẫn `received_at`? Nếu chỉ giữ một thì mất khả năng gì?

12 Vì sao Redis pub/sub bị cấm trên đường đi của giao dịch nhưng được dùng cho giao diện?

13 Khách dùng "Wallet ID card". Ai đã thu tiền? CSMS được phép làm gì và không được làm gì?

14 Tài xế chọn sạc theo số tiền. Ai chịu trách nhiệm dừng phiên, và chuyện gì xảy ra nếu CSMS mất kết nối đúng lúc đó?

07

Chín câu dễ viết sai — kiểm lại trước khi đưa vào Report

ĐỪNG VIẾT	SỰ THẬT
"Hệ thống đã được chứng nhận OCPP"	Chạy qua kịch bản OCTT **không phải** chứng nhận. Chứng nhận cần test lab độc lập của OCA và form PICS
"SteVe đã được OCA chứng nhận"	Chứng chỉ thuộc về Powerfill — một sản phẩm thương mại phái sinh. Nó **không lan sang** SteVe mã nguồn mở
"CitrineOS compliant OCPP 1.6"	Là marketing tự khai, không phải chứng nhận
"plug-and-charge của trụ = ISO 15118"	Sai hoàn toàn. Gọi là **free vend**
"backoff exponential"	Spec ghi **tuyến tính**. Nhiều nguồn thứ cấp chép sai
"`meterStart` tính bằng kWh"	**Wh.** Nhầm là sai hoá đơn 1.000 lần
"EVerest đã chết"	Chỉ repo `EVerest/libocpp` bị archive 27/04/2026. `everest-core` vẫn hoạt động
"MaEVe dùng EVerest làm simulator"	Claim này đã bị bác 0–3. Không viết
Fork hoặc copy code SteVe	Giấy phép chưa chốt. Chạy SteVe như tiến trình riêng thì không phát sinh câu hỏi pháp lý

> Thêm một ràng buộc giấy phép: thư viện `ocpp-rpc` là MIT, **nhưng bộ schema OCA đi kèm là CC BY-ND 4.0** — dùng nguyên văn, không được sửa.

08

Đọc tiếp ở đâu, và những gì chưa ai biết

Trong repo

TỆP	ĐỌC KHI NÀO
`docs/WP2-KIEN-TRUC-TECHSTACK.md`	Sau tài liệu này. Chi tiết đầy đủ 11 ADR, mô hình dữ liệu, hợp đồng với WP3 / 4 / 5
`docs/WP2-LO-TRINH-KY-THUAT.md`	Song song với mục 05. Đó là bản thực hành theo ngày *(lưu ý: neo theo lịch cũ, ngày tháng cần dời)*
`docs/NGHIEN-CUU-NEN-TANG.md`	Khi cần nguồn cho một khẳng định — 21 claim đã kiểm chứng đối kháng
`docs/PHAN-TICH-TAI-LIEU-WP1.md`	Trước họp nhóm
`ocppdl/`	Tra JSON schema của từng bản tin. **Chỉ để tra, không phải dependency**

TỆP	ĐỌC KHI NÀO
`CONTRIBUTING.md`	Trước PR đầu tiên: nhánh `wp2/be-04-…`, Conventional Commits kèm mã task, PR < 400 dòng, một approve + CI xanh

Ngoài repo

- **OCPP 1.6 Specification + JSON schema** — bản gốc từ Open Charge Alliance. Nguồn duy nhất đáng trích dẫn về hành vi giao thức
- **Manual gốc D1 (.docx) và YJ-EU (.pdf)** — trong Google Drive dự án, thư mục `Context\`. Nguồn duy nhất về hành vi phần cứng
- **Mã nguồn `ocpp-rpc`** — đọc trực tiếp, đừng đọc README

> ### Bốn câu chỉ hãng Ruihua trả lời được — simulator vô dụng ở đây
>
> 1. Khi dùng thẻ ví, trụ có gửi số tiền đã trừ hoặc số dư lên CSMS không? Qua `DataTransfer`?
> 2. Phiên free vend gửi `idTag` với giá trị gì?
> 3. Firmware hỗ trợ những khoá `ChangeConfiguration` nào?
> 4. Firmware thật sự gửi những `measurand` nào trong `MeterValues`?

Câu 1 ảnh hưởng trực tiếp tới ADR-0012 và tới phạm vi phải khoá ở Tuần 6. Thời gian chờ hãng trả lời không kiểm soát được — **gửi sớm**.

> ### Rủi ro R-A02 — lý do tài liệu này viết theo lối giải thích
>
> Rủi ro đã ghi trong sổ của nhóm: *dựa vào code hoặc tài liệu do AI sinh ra mà không hiểu, rồi hỏng khi hội đồng truy vì sao nó chạy.* CSMS là phần bị đào sâu nhất trong buổi bảo vệ.
>
> Phép thử đơn giản cho mọi dòng code và mọi câu trong Report: **giải thích được bằng lời cho một người không đọc tài liệu này không?** Nếu không, chưa xong.
